Control Statements: Part 2

Visual C# How to Program, 6/e

Essentials of Counter-Controlled Iteration

- Counter-controlled iteration requires
  - a **control variable** (or loop counter)
  - the **initial value** of the control variable
  - the **increment** (or **decrement**) by which the control variable is modified each **iteration of the loop**
  - the **loop-continuation condition**

```
1  // Fig. 6.1: WhileCounter.cs
2  // Counter-controlled iteration with the while iteration statement.
3  using System;
4
5  class WhileCounter
6  {
7     static void Main()
8     {
9        int counter = 1; // declare and initialize control variable
10
11       while (counter <= 10) // loop-continuation condition
12       {
13          Console.Write($"{counter}  ");
14          ++counter; // increment control variable
15       }
16
17       Console.WriteLine();
18    }
19 }
```

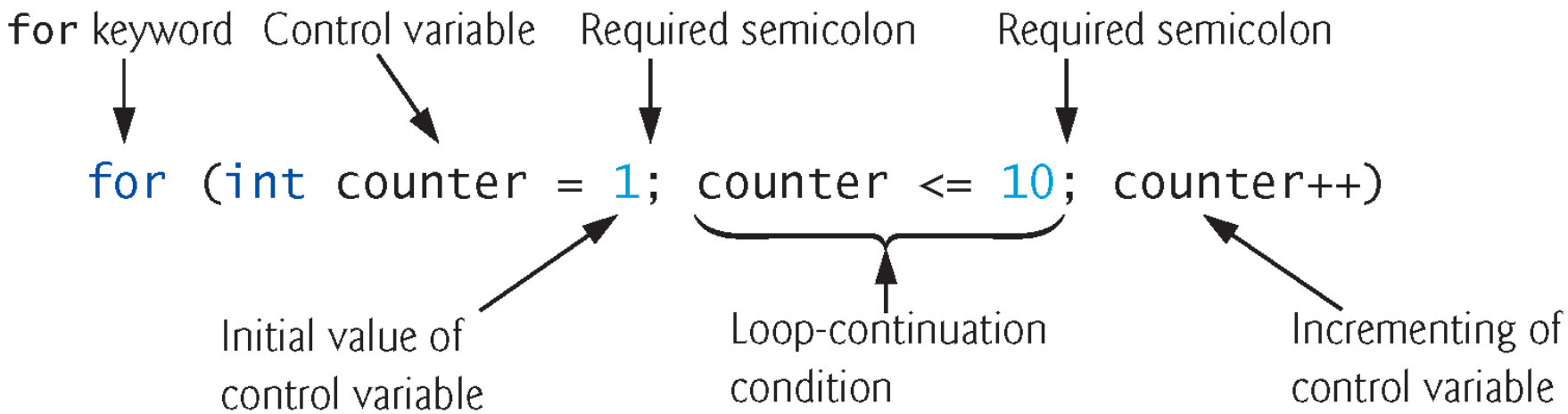
```
1  2  3  4  5  6  7  8  9  10
```

for Iteration Statement

- The **for iteration statement** specifies the elements of counter-controlled-iteration in a single line of code.

```
1  // Fig. 6.2: ForCounter.cs
2  // Counter-controlled iteration with the for iteration statement.
3  using System;
4
5  class ForCounter
6  {
7     static void Main()
8     {
9        // for statement header includes initialization,
10       // loop-continuation condition and increment
11       for (int counter = 1; counter <= 10; ++counter)
12       {
13          Console.Write($"{counter}  ");
14       }
15
16       Console.WriteLine();
17    }
18 }
```

```
1  2  3  4  5  6  7  8  9  10
```



`for` Iteration Statement (Cont.)

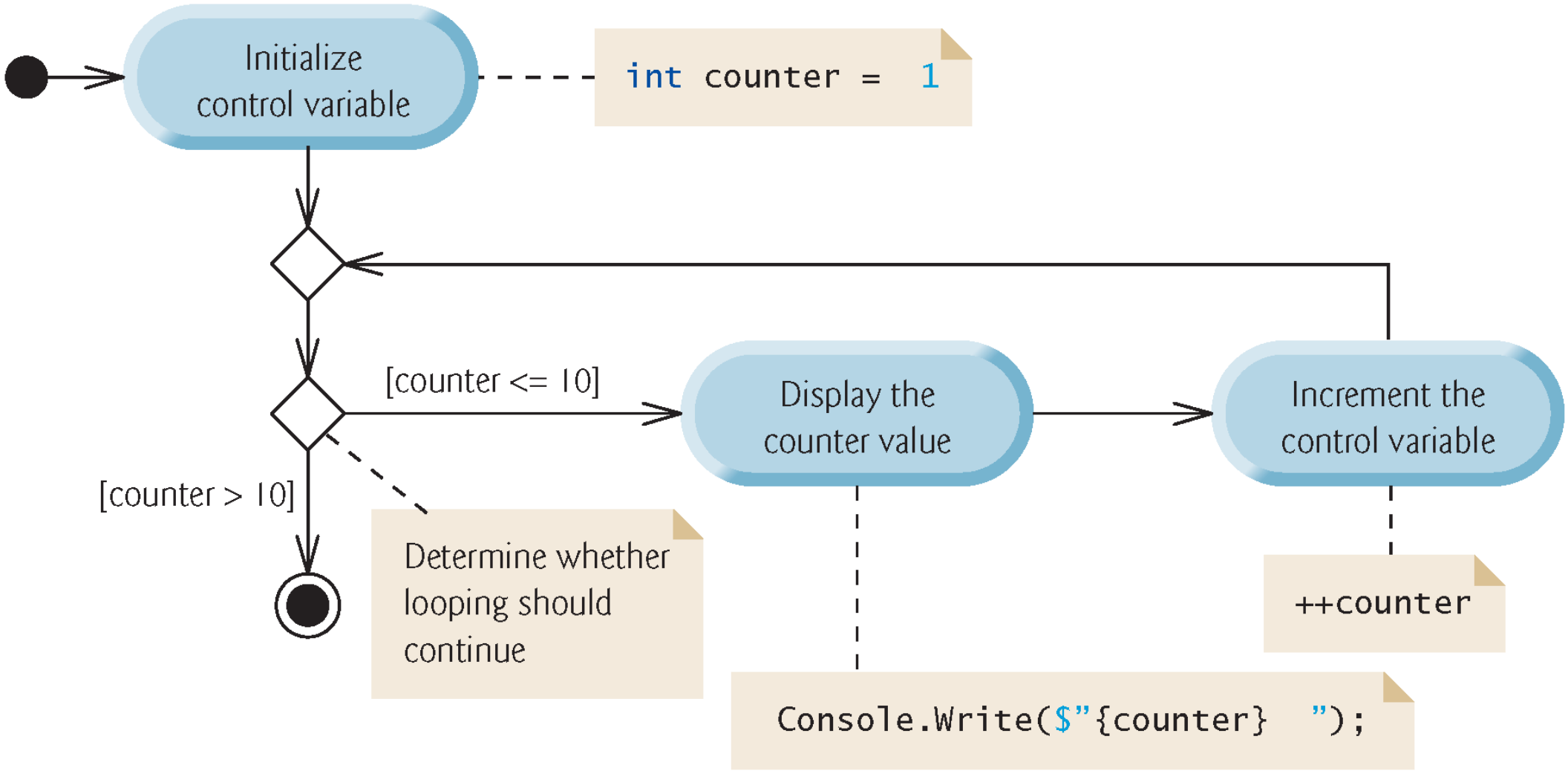
- The initialization, loop-continuation condition and increment portions of a `for` statement can contain arithmetic expressions.
- For example, assume that `x = 2` and `y = 10`; if `x` and `y` are not modified in the body of the loop, then the statement

```
for (int j = x; j <= 4 * x * y; j += y / x)
```

  is equivalent to the statement

```
for (int j = 2; j <= 80; j += 5)
```

- If the loop-continuation condition is initially `false`, the app does not execute the `for` statement's body.



App: Summing Even Integers

- The app in Fig. uses a `for` statement to sum the even integers from 2 to 20 and store the result in an `int` variable called `total`.

```
1  // Fig. 6.5: Sum.cs
2  // Summing integers with the for statement.
3  using System;
4
5  class Sum
6  {
7     static void Main()
8     {
9        int total = 0; // initialize total
10
11       // total even integers from 2 through 20
12       for (int number = 2; number <= 20; number += 2)
13       {
14          total += number;
15       }
16
17       Console.WriteLine($"Sum is {total}"); // display results
18    }
19 }
```

```
Sum is 110
```

App: Compound-Interest Calculations

- A person invests $1,000 in a savings account yielding 5% interest, compounded yearly. Calculate and display the amount of money in the account at the end of each year for 10 years.

  $a = p\ (1 + r\ )^n$

  p is the original amount invested (i.e., the principal)
  r is the annual interest rate (use 0.05 for 5%)
  n is the number of years
  a is the amount on deposit at the end of the nth year.

Fig. uses a loop that performs the calculation for each of the 10 years the money remains on deposit.

```
1  // Fig. 6.6: Interest.cs
2  // Compound-interest calculations with for.
3  using System;
4
5  class Interest
6  {
7     static void Main()
8     {
9        decimal principal = 1000; // initial amount before interest
10       double rate = 0.05; // interest rate
11
12       // display headers
13       Console.WriteLine("Year   Amount on deposit");
14
```

```
15          // calculate amount on deposit for each of ten years
16          for (int year = 1; year <= 10; ++year)
17          {
18             // calculate new amount for specified year
19             decimal amount = principal *
20                ((decimal) Math.Pow(1.0 + rate, year));
21
22             // display the year and the amount
23             Console.WriteLine($"{year,4}{amount,20:C}");
24          }
25       }
26    }
```

```
Year   Amount on deposit
   1           $1,050.00
   2           $1,102.50
   3           $1,157.63
   4           $1,215.51
   5           $1,276.28
   6           $1,340.10
   7           $1,407.10
   8           $1,477.46
   9           $1,551.33
  10           $1,628.89
```

Fig. 6.6 | Compound-interest calculations with `for`. (Part 3 of 3.)

Performing the Interest Calculations with `Math` Method `pow`

- Many classes provide static methods that cannot be called on objects—they must be called using a class name.
- You can call a `static` method by specifying the class name followed by the member access (`.`) operator and the method name:

  ClassName.*methodName*(*arguments*)

- `Math.Pow(`*x*`, `*y*`)` calculates the value of *x* raised to the *y*th power.

Formatting with Field Widths and Alignment

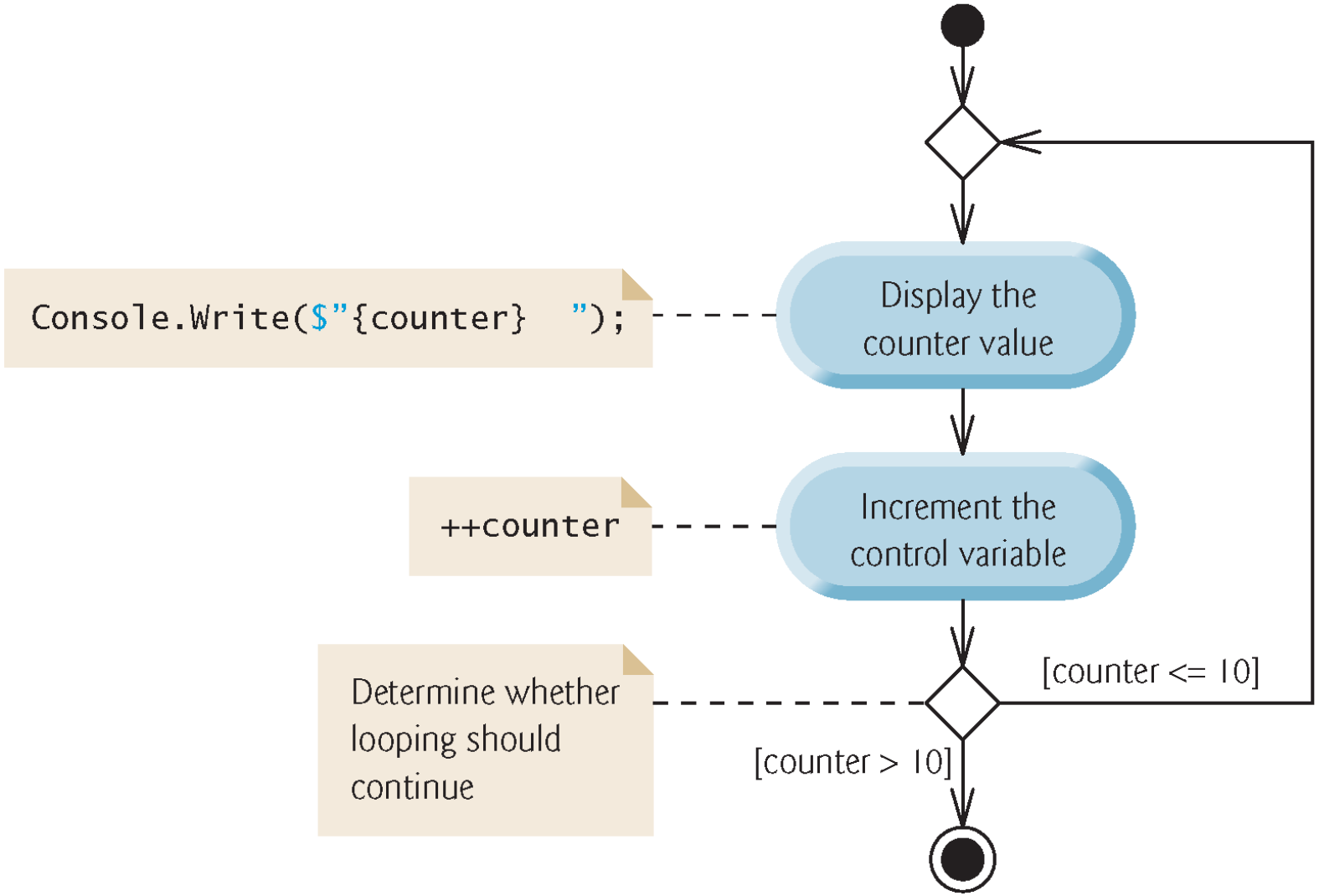
- `Console.WriteLine($"{year,4}{amount,20:C}");`
  - displays the year and the amount on deposit at the end of that year.
- The interpolation expression
  - `{year,4}`
- formats the year.
  - The integer 4 after the comma indicates that the value output should be displayed with a field width of 4.
  - If the value to be output is fewer than four character positions wide (one or two characters in this example), the value is right-aligned in the field by default.
  - If the value to be output were more than four character positions wide, the field width would be extended to the right to accommodate the entire value.

`do...while` Iteration Statement

- The **`do...while` iteration statement** is similar to the `while` statement.

```
1  // Fig. 6.7: DoWhileTest.cs
2  // do...while iteration statement.
3  using System;
4
5  class DoWhileTest
6  {
7     static void Main()
8     {
9        int counter = 1; // initialize counter
10
11       do
12       {
13          Console.Write($"{counter}  ");
14          ++counter;
15       } while (counter <= 10); // required semicolon
16
17       Console.WriteLine();
18    }
19 }
```

```
1  2  3  4  5  6  7  8  9  10
```



`switch` Multiple-Selection Statement

- The **`switch` multiple-selection** statement performs different actions based on the value of an expression.
- Each action is associated with the value of a **constant integral expression** or a **constant string expression** that the expression may assume.

```
1  // Fig. 6.9: LetterGrades.cs
2  // Using a switch statement to count letter grades.
3  using System;
4
5  class LetterGrades
6  {
7     static void Main()
8     {
9        int total = 0; // sum of grades
10       int gradeCounter = 0; // number of grades entered
11       int aCount = 0; // count of A grades
12       int bCount = 0; // count of B grades
13       int cCount = 0; // count of C grades
14       int dCount = 0; // count of D grades
15       int fCount = 0; // count of F grades
16
```

```
17          Console.WriteLine("Enter the integer grades in the range 0-100.");
18          Console.WriteLine(
19             "Type <Ctrl> z and press Enter to terminate input:");
20
21          string input = Console.ReadLine(); // read user input
22
23          // loop until user enters the end-of-file indicator (<Ctrl> z)
24          while (input != null)
25          {
26             int grade = int.Parse(input); // read grade off user input
27             total += grade; // add grade to total
28             ++gradeCounter; // increment number of grades
29
```

```
30            // determine which grade was entered
31            switch (grade / 10)
32            {
33               case 9: // grade was in the 90s
34               case 10: // grade was 100
35                  ++aCount; // increment aCount
36                  break; // necessary to exit switch
37               case 8: // grade was between 80 and 89
38                  ++bCount; // increment bCount
39                  break; // exit switch
40               case 7: // grade was between 70 and 79
41                  ++cCount; // increment cCount
42                  break; // exit switch
43               case 6: // grade was between 60 and 69
44                  ++dCount; // increment dCount
45                  break; // exit switch
46               default: // grade was less than 60
47                  ++fCount; // increment fCount
48                  break; // exit switch
49            }
50
51            input = Console.ReadLine(); // read user input
52         }
```

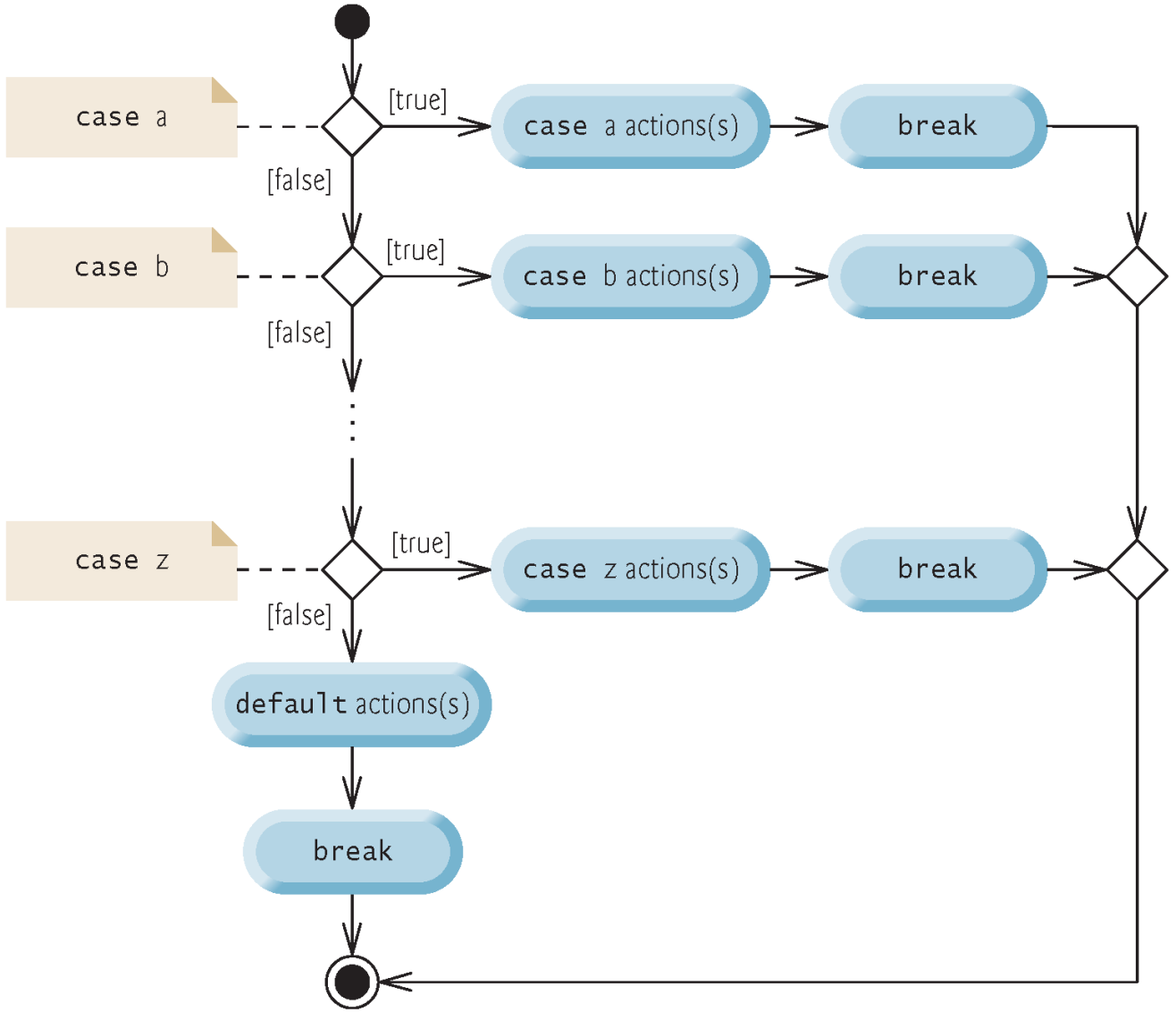
```
53
54          Console.WriteLine("\nGrade Report:");
55
56          // if user entered at least one grade...
57          if (gradeCounter != 0)
58          {
59             // calculate average of all grades entered
60             double average = (double) total / gradeCounter;
61
62             // output summary of results
63             Console.WriteLine(
64                $"Total of the {gradeCounter} grades entered is {total}");
65             Console.WriteLine($"Class average is {average:F}");
66             Console.WriteLine("Number of students who received each grade:");
67             Console.WriteLine($"A: {aCount}"); // display number of A grades
68             Console.WriteLine($"B: {bCount}"); // display number of B grades
69             Console.WriteLine($"C: {cCount}"); // display number of C grades
70             Console.WriteLine($"D: {dCount}"); // display number of D grades
71             Console.WriteLine($"F: {fCount}"); // display number of F grades
72          }
```

```
73          else // no grades were entered, so output appropriate message
74          {
75             Console.WriteLine("No grades were entered");
76          }
77       }
78    }
```

```
Enter the integer grades in the range 0-100.
Type <Ctrl> z and press Enter to terminate input:
99
92
45
57
63
71
76
85
90
100
^Z

Grade Report:
Total of the 10 grades entered is 778
Class average is 77.80
Number of students who received each grade:
A: 4
B: 1
C: 2
D: 1
F: 2
```

Fig. 6.9 | Using a `switch` statement to count letter grades. (Part 6 of 6.)



break Statement

- The `break` statement causes immediate exit from a statement.

```
1  // Fig. 6.13: BreakTest.cs
2  // break statement exiting a for statement.
3  using System;
4
5  class BreakTest
6  {
7     static void Main()
8     {
9        int count; // control variable also used after loop terminates
10
11       for (count = 1; count <= 10; ++count) // loop 10 times
12       {
13          if (count == 5) // if count is 5,
14          {
15             break; // terminate loop
16          }
17
18          Console.Write($"{count} ");
19       }
20
21       Console.WriteLine($"\nBroke out of loop at count = {count}");
22    }
23 }
```

`continue` Statement

- The **`continue` statement** skips the remaining statements in the loop body and tests whether to proceed with the next iteration of the loop.
- In a `for` statement, the increment expression executes, then the app evaluates the loop-continuation test.

```
1  // Fig. 6.14: ContinueTest.cs
2  // continue statement skipping an iteration of a for statement.
3  using System;
4
5  class ContinueTest
6  {
7     static void Main()
8     {
9        for (int count = 1; count <= 10; ++count) // loop 10 times
10       {
11          if (count == 5) // if count is 5,
12          {
13             continue; // skip remaining code in loop
14          }
15
16          Console.Write($"{count} ");
17       }
18
19       Console.WriteLine("\nUsed continue to skip displaying 5");
20    }
21 }
```

Logical Operators

- Simple conditions are expressed in terms of the relational operators >, <, >= and <=, and the equality operators == and !=.
  - Each expression tests only one condition.
- C# provides **logical operators** to enable you to form more complex conditions by combining simple conditions.
- The logical operators are && (conditional AND), || (conditional OR), & (boolean logical AND), | (boolean logical inclusive OR), ^ (boolean logical exclusive OR) and ! (logical negation).

Conditional AND (&&) Operator

- The **&&** (**conditional AND**) operator works as follows:

```
if (gender == 'F' && age >= 65)
{
   ++seniorFemales;
}
```

- The combined condition is true if and only if *both* simple conditions are true.
- The combined condition may be more readable with redundant parentheses:

```
(gender == 'F') && (age >= 65)
```

Conditional AND (&&) Operator

- The table shows all four possible combinations of `false` and `true` values for *expression1* `and` *expression2*.
- Such tables are called **truth tables**.

expression1	expression2	expression1 && expression2
false	false	false
false	true	false
true	false	false
true	true	true

Conditional OR (||) Operator

- The || (**conditional OR**) operator, as in the following app segment:

```
if ((semesterAverage >= 90) || (finalExam >= 90))
{
   Console.WriteLine ("Student grade is A");
}
```

- The complex condition evaluates to `true` if either or both of the simple conditions are true.
- Figure is a truth table for operator conditional OR (||).

expression1	expression2	expression1 || expression2
false	false	false
false	true	true
true	false	true
true	true	true

Short-Circuit Evaluation of Complex Conditions

- The parts of an expression containing && or || operators are evaluated only until it's known whether the condition is true or false.
  - `(gender == 'F') & (age >= 65)`
- If `gender` is not equal to `'F'` (i.e., it is certain that the entire expression is `false`) evaluation stops.
- This feature is called **short-circuit evaluation**.

Boolean Logical AND (&) and Boolean Logical OR (|) Operators

- The **boolean logical AND (&)** and **boolean logical inclusive OR (|)** operators do not perform short-circuit evaluation.
- This is useful if the right operand has a required **side effect**. For example:

```
(birthday == true) | (++age >= 65)
```

- This ensures that the condition `++age >= 65` will be evaluated.

Boolean Logical Exclusive OR (^)

- A complex condition containing the **boolean logical exclusive OR** (^) operator (also called the **logical XOR operator**) is true *if and only if one of its operands is* `true` *and the other is* `false`.
- Figure is a truth table for the boolean logical exclusive OR operator (^).

expression1	expression2	expression1 ^ expression2
false	false	false
false	true	true
true	false	true
true	true	false

Logical Negation (!) Operator

- The ! (**logical negation** or **not**) operator enables you to "reverse" the meaning of a condition.
  - ```
    if (! (grade == sentinelValue))
    {
        Console.WriteLine($"The next grade is {grade}");
    }
    ```
- In most cases, you can avoid using logical negation by expressing the condition differently:
  - ```
    if (grade != sentinelValue)
    {
        Console.WriteLine($"The next grade is {grade}");
    }
    ```
- Figure is a truth table for the logical negation operator.

expression	!expression
`false`	`true`
`true`	`false`

Operators	Associativity	Type
`.` `new` `++`*(postfix)* `--`*(postfix)*	left to right	highest precedence
`++` `--` `+` `-` `!` (*type*)	right to left	unary prefix
`*` `/` `%`	left to right	multiplicative
`+` `-`	left to right	additive
`<` `<=` `>` `>=`	left to right	relational
`==` `!=`	left to right	equality
`&`	left to right	boolean logical AND
`^`	left to right	boolean logical exclusive OR
`|`	left to right	boolean logical inclusive OR
`&&`	left to right	conditional AND
`||`	left to right	conditional OR
`?:`	right to left	conditional
`=` `+=` `-=` `*=` `/=` `%=`	right to left	assignment